Towards Unified File Management for Data Lakehouse Systems

Keonwoo Oh
University of Maryland
College Park, Maryland, USA

Akshay Mehra
Dolby Labs.
Atlanta, Georgia, USA

Joshua Kimball
Dolby Labs.
Atlanta, Georgia, USA

Amol Deshpande
University of Maryland
College Park, Maryland, USA

arXiv:2611.15975v1 [cs.DB] 11 Nov 2026

Abstract

Data lakehouse systems have garnered widespread adoption, yet their metadata management systems face a fundamental architectural dilemma. File-based OTFs (Open Table Formats) scale well, but suffer from metadata bloat, high commit latencies, the absence of multi-table transactions, and unsafe garbage collection. On the other hand, catalogs relying entirely on a transactional database incur severe scalability bottlenecks due to analytic file-level metadata. To reconcile this tension, we present Raven, an operational catalog that stores transactional metadata within a database while offloading analytical metadata to underlying OTF structures. Based on this hybrid architecture, Raven introduces an extended Merge-on-Read (MoR) mechanism to reduce commit latency and mitigate metadata bloat, alongside multi-table transactions under snapshot isolation. Furthermore, Raven unifies core data lakehouse semantics including versioning and reference tracking under an extensible ER (Entity-Relationship) metadata model, enabling safe global garbage collection across diverse types of assets. Empirical evaluations demonstrate that Raven substantially reduces commit latency and metadata bloat when integrated with Apache Iceberg and Delta Lake, maintains a catalog storage footprint orders of magnitude smaller than database-only systems like DuckLake, and achieves scalable garbage collection with minimal disruption. [1]

CCS Concepts

• **Information systems** → **Data management systems**.

Keywords

Catalog, Data Lakehouse, Open Table Format, Garbage Collection, Multi-Table Transactions

1 Introduction

Historically, big data processing engines and traditional data warehouses served distinct roles [61]. However, the data lakehouse architecture, designed to combine the functionalities of both system types, has become more commonplace recently [2, 9]. Unlike traditional systems that manage data in isolation, a data lakehouse achieves a composable data stack where multiple engines operate directly on shared datasets in scalable storage (e.g., cloud object stores) using standard formats like Apache Parquet [68] and ORC [67] (Figure 1). Simultaneously, it aims to support traditional warehouse functions such as transactions, data maintenance, and access control.

[1] Code is available at: https://github.com/umddb/raven

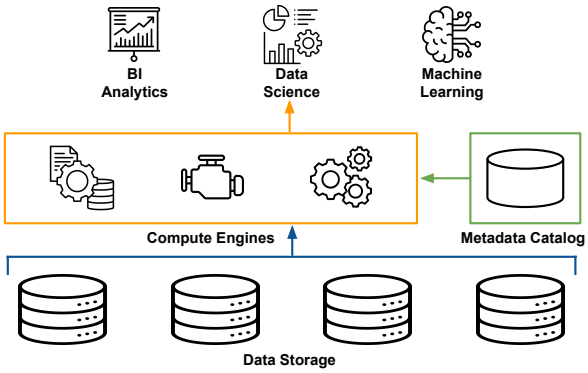

Figure 1: An example data lakehouse architecture, characterized by disaggregated components/subsystems.

Increasing adoption of the data lakehouse architecture is closely tied to the emergence of advanced metadata systems. On the one hand, there are lakehouse storage formats, also known as OTFs (Open Table Formats), including Delta Lake [71], Apache Iceberg [65], and Apache Hudi [63]; these specify how to store the metadata of a tabular dataset, such as the table schema, partitioning scheme, list of data files, and file-level statistics, as a set of files alongside the data. On the other hand, a data lakehouse also requires a separate catalog service that provides a mapping from the logical namespace (e.g., database and table names) to the file directory path(s) as well as data governance and access control functions.

Despite growing popularity, data lakehouses face significant limitations in their metadata systems. We examine the traditional data warehouses and earlier big data platforms before discussing the limitations of data lakehouse metadata systems.

1.1 Data Warehouses and Big Data Systems

Commercial data warehouses, including Snowflake [20], Google BigQuery [38], Amazon Redshift [11], Microsoft Fabric [42], ClickHouse [55], and SingleStore [57], typically rely on transactional systems or distributed coordination services for metadata management. For example, Snowflake uses FoundationDB [79], a highly scalable transactional key-value store, for metadata storage [45]. However, closed architectures leave many advanced features unavailable for external data assets [29, 59].

On the other hand, big data processing engines such as Hive [74], Spark [10], Presto/Trino [56], Impala [66], and Flink [15] have long used the HMS (Hive Metastore) as their primary operational catalog. An HMS instance consists of a Thrift server, a DataNucleus

Table 1: PostgreSQL storage size for DuckLake when scaling integer column count in a single table with 100,000 data files.

Column Count	10	100	1,000	10,000
Metadata Storage (GB)	0.15	1.14	10.97	109.31

Table 2: Metadata placement across three representative data lakehouse metadata systems.

Metadata Type	HMS + Iceberg	Raven + Iceberg	DuckLake
Table Name	Database	Database	Database
OTF Root Path	Database	Database	N/A
Table Versioning	Object Store	Database	Database
Table Schema	Object Store	Database	Database
File Visibility	Object Store	Database	Database
File Statistics	**Object Store**	**Object Store**	**Database**

ORM (Object-Relational Mapping) layer, and an RDBMS backend. Because the RDBMS maintains only coarse-grained mappings from tables or partitions to directory paths, query engines must retrieve these paths from HMS and issue *list()* calls against the file system (cloud object store) to locate data files. For large tables with millions of files in cloud object stores, this introduces severe latency bottlenecks [8]. In addition, lacking file-level metadata, HMS does not support transactional guarantees under concurrent operations.

1.2 Open Table Formats and Hidden Cost of Scalability

OTFs, including Delta Lake, Apache Iceberg, and Apache Hudi, were created to address the limitations of HMS. By materializing file-level metadata as physical files alongside data, OTFs avoid storage system's expensive *list()* operations. They also support single-table ACID transactions based on OCC (Optimistic Concurrency Control) mechanisms and advanced features such as time-travel, branching/cloning, schema evolution, and maintenance operations. Meanwhile, modern catalog systems, such as Unity Catalog [16], Polaris [58], OpenHouse [54], OneHouse [51], and Nessie [53] have emerged to replace HMS. However, like HMS, these catalogs store a pointer to the root directory or metadata file for each table and delegate file-level metadata tracking to the underlying OTFs.

Delta Lake and Hudi use log-based mechanisms to record file additions, deletions, and schema changes in ordered log files, reconstructing snapshots by replaying logs from checkpoints. On the other hand, Iceberg uses a hierarchical structure: a root metadata JSON file points to manifest list Avro files, which in turn reference manifest Avro files containing individual data file paths and file-level statistics.

The guiding design principle behind OTFs is that metadata is no longer small [27, 62]. To manage scalability, they offload most metadata as files into cloud object store, ignoring that many metadata operations remain transactional and advanced ACID guarantees are necessary to support complex features with data integrity. We illustrate these limitations using Iceberg's *appendFiles()* method. Even adding a single data file (any data modification) results in three new nested metadata files: a manifest file, a manifest list file, and a root metadata file, causing severe bloat. The commit protocol concludes with a single compare-and-swap (CAS) operation on the catalog reference to the root metadata file, inherently restricting ACID guarantees to a single table. This process incurs high commit latency due to multiple file writes and the final CAS operation.

We formalize the main limitations of OTFs as follows:

(1) **Small Files Problem:** Frequent transactional updates (e.g., from streaming data) can create a large number of small metadata files, degrading read performance and straining the storage system's metadata layer [1].
(2) **High Commit Latency:** The commit protocol involves writing new metadata files, followed by a final CAS operation, which together introduce overhead due to high latencies of cloud object stores [33, 49].
(3) **No Multi-table Transactions:** OTFs support only single-table transactions as they rely on a single CAS operation on a table-level object for the final commit [8, 64].
(4) **Incorrect File Deletion:** Updates, compaction, etc., can result in obsolete files, making garbage collection essential, but because metadata is stored as files in cloud object store, reference tracking is expensive and difficult to represent. Current systems fail to keep track of multiple access paths, so files in active use may be deleted (Section 4.1).

Ecosystem fragmentation presents another major challenge as each OTF relies on ad-hoc workarounds and custom semantics to support similar features like transactions, branching, and garbage collection. Consequently, as feature sets expand, system complexity grows rapidly.

1.3 Centralization and Big Metadata Problem

Recent systems addressing these issues, such as DuckLake [26] and TreeCat [49], represent the opposite extreme by storing all metadata in a transactional database. DuckLake utilizes an RDBMS (e.g., PostgreSQL, MySQL, or DuckDB) as its backend, whereas TreeCat relies on a custom hierarchical transactional store. While these systems resolve the above OTF limitations, they encounter scalability bottlenecks already observed in industry. For example, Google BigQuery, despite using Spanner, had to offload a subset of file-level metadata into a BigQuery table [27]. Databricks recently reported compressed metadata for a single table reaching up to 1 TB [77].

The primary culprit behind this scalability bottleneck is file-level statistics, which are necessary for cost-based query optimization and data pruning. These statistics track at least four values (row count, null count, minimum, and maximum) per column per file, increasing linearly with the product of column count and file count. For wide tables (1,000s of columns) spanning millions of files, this creates a severe scalability challenge. We empirically verify this by measuring DuckLake's metadata footprint (RDBMS size) across increasing column counts for a table with 100,000 data files (Table 1). At 10,000 columns, the footprint reaches 109.31 GB, with statistics accounting for over 99.9% of it. Across thousands of tables, maintaining the cumulative metadata inside a transactional store becomes difficult to scale.

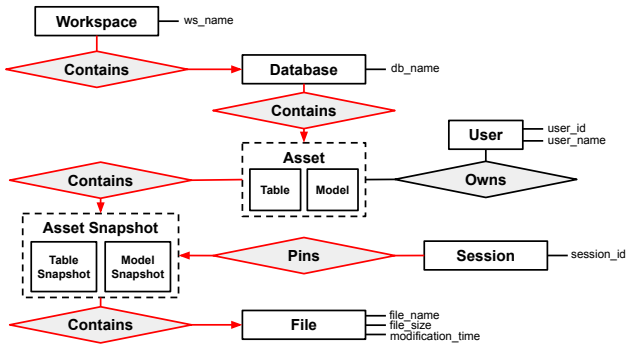

Figure 2: An example metadata schema in the ER representation. Referential relationships (Section 4.3) are colored in red, and abstract entities are marked by dashed boundary lines.

1.4 Raven: An Extensible Hybrid System

Metadata operations inherently encompass both **transactional** and **analytical** workloads. Hence, relying exclusively on either cloud object storage or a transactional database is suboptimal. At the same time, unifying common functionality while preserving extensibility is vital for long-term system evolution. In this paper, we present Raven, a standalone catalog system with two main contributions:

(1) **Hybrid Architecture:** Raven pulls the subset of metadata requiring transactional guarantees into a database, while retaining most metadata in standard OTFs for scalability (Table 2). We implement two features, extended Merge-on-Read (MoR) and multi-table transactions, that address the first three OTF limitations in Section 1.2 while avoiding main scalability bottlenecks of a database-only design.

(2) **Unified Extensible Interface:** We unify core lakehouse semantics, such as versioning and reference tracking, through an ER metadata model that enables safe global garbage collection (addressing the last OTF limitation). Anticipating future lakehouse formats (not necessarily tabular data), Raven provides an extensible abstraction designed to generalize across diverse asset types.

In the next section, we describe Raven's core metadata model and system interface.

2 Metadata Model and System Interface

Storing critical metadata in a unified transactional store requires balancing control and extensibility. Excessive control results in a rigid system, while too little fragments the ecosystem with redundancies. To address this, we introduce three important constructs. We now cover the main ER model and the `FILELIST` construct, while deferring the discussion of garbage collection semantics to Section 4.3.

2.1 Entity-centric Abstraction

In addition to file-level metadata, catalog metadata includes logical entities (e.g., databases, tables, and users) and their relationships. To represent them, we adopt a modified ER (Entity-Relationship) model [17]. We introduce a DSL (Domain-Specific Language) called ERQL (Entity-Relationship Query Language) with a unified syntax: a DDL (Data Definition Language) for defining and extending schemas, and a DML (Data Manipulation Language) for catalog operations. Below is DDL for the schema in Figure 2.

```
CREATE ENTITY Workspace(ws_name VARCHAR(255));
CREATE ENTITY Database(db_name VARCHAR(255));
CREATE RELATIONSHIP Contains(Workspace, Database);
```

A highly demanded extension for the catalog is the registration of non-tabular asset types, such as machine learning models and multimedia data. To support this, we introduce the `ABSTRACT ENTITY` type. It specifies the `RELATIONSHIP` types available to its derived `ENTITY` types while intentionally leaving the attribute schema unconstrained. As shown below, distinct asset types can be defined as derived entities of the `Asset ABSTRACT ENTITY` with their own attribute schemas while inheriting shared relationships like `Contains(Database, Asset)` as shown below.

```
CREATE ABSTRACT ENTITY Asset();
CREATE RELATIONSHIP Contains(Database, Asset);
CREATE ENTITY Table (...) IMPLEMENTS Asset;
CREATE ENTITY Model (...) IMPLEMENTS Asset;
```

In the system execution path, the DML layer sits between the top-level catalog APIs and the underlying storage backends. We illustrate how catalog operations can be mapped to the ERQL DML. Consider *getTable()*, which retrieves the table-level metadata for a specific workspace, database, and table name. The operation can be directly mapped to the following parameterized ERQL query:

```
SELECT t.*
FROM Contains(Workspace w, Database d),
     Contains(d, Table t)
WHERE w.ws_name=$1 AND d.db_name=$2 AND t.name=$3;
```

Each entity is uniquely identified by its type and object ID and can also be queried by the object ID.

2.2 FileList Entity

To address the scalability challenges of file-level metadata, we introduce the `FILELIST` construct, which models any asset as a versioned collection of files. It addresses two primary challenges: (1) storing file-level statistics (Section 1.3), and (2) tracking file references as standard relationships in the ER model. For example, even when a new asset snapshot differs only slightly from its predecessor, a naive ER representation must create millions of duplicate file references.

We resolve the first challenge by separating transactional and analytical file-level metadata. `FILELIST` retains only the essential attributes required for transactional correctness (e.g., file path and format), while offloading file-level statistics, unnecessary for determining file visibility and lifetime, to underlying OTF structures. Meanwhile, the catalog transactionally tracks the OTF metadata files themselves within the `FILELIST`, ensuring **all files** remain under its control. To resolve the second challenge, we avoid modeling every snapshot-to-file association as an explicit ER relationship, relying instead on the constrained relational representation of `FILELIST`.

`FILELIST` represents a collection of files, uniquely identified by an object ID and a version ID. A chain of `FILELIST` versions sharing the same object ID forms a snapshot history, with each

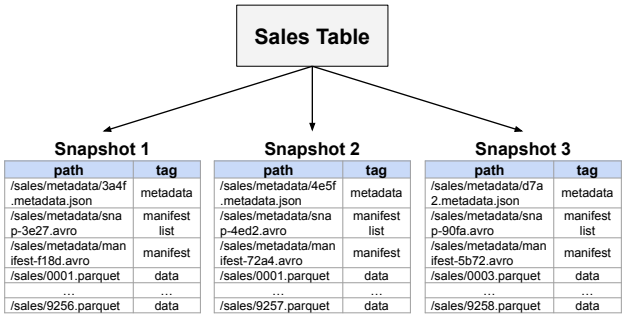

Figure 3: Consecutive Iceberg table snapshots are represented as FILELIST versions. A few attributes of the FILELIST view are omitted for brevity.

version referencing its file collection. For example, a table snapshot containing all files visible at a given time can be declared a `FILELIST` entity type:

```
CREATE FILELIST ENTITY TableSnapshot (table_name,
     create_time,...);
```

The collection of files is represented as a **relational view** with a fixed schema containing only basic file attributes such as file path, format, size, modification time, and tag.

As an example, we show how an Iceberg table is represented as a `FILELIST` in Raven. Although an Iceberg snapshot consists of a hierarchy of metadata and data files (Section 1.2), `FILELIST` provides a flat relational view of all constituent files regardless of internal semantics (Figure 3). Each `FILELIST` version captures a specific snapshot, referencing new metadata and data files while sharing unmodified files with other versions. For example, the query below retrieves all data files for object ID 13 and version ID 2:

```
SELECT file_path, file_size, format
FROM FILELIST SNAPSHOT TableSnapshot(13, 2)
WHERE tag = 'data';
```

The client can also retrieve the list of manifest files to apply file pruning based on file-level statistics as follows.

```
SELECT file_path
FROM FILELIST SNAPSHOT TableSnapshot(13, 2)
WHERE tag = 'manifest';
```

Common data lakehouse operations such as CDC (Change Data Capture) extraction and in-memory snapshot update require identifying changes between versions. To this end, we provide `DELTA` keyword for retrieving file-level additions and deletions between two snapshots. The query below demonstrates retrieving the delta between versions 2 and 3:

```
SELECT action, file_path, tag
FROM FILELIST DELTA TableSnapshot(13, 2, 3);
```

Raven is oblivious to the specific formats and types of files that a `FILELIST` contains, allowing the system to be extended for new types of assets and storage formats in the future.

3 System Architecture

In this section, we present Raven's system architecture and introduce two important features of Raven that address a subset of the OTF problems, including metadata bloat, high commit latencies, and the absence of multi-table transactions. Raven consists of three main components (Figure 4):

(1) **Catalog Service:** This is the front-end server that handles requests from various clients via gRPC.
(2) **Metadata Store:** The metadata store is the main backend data system that stores the metadata accessed by the catalog service and the garbage collector.
(3) **Garbage Collector** (Section 4)**:** The garbage collector periodically retrieves updates from the metadata store, identifies unreachable metadata objects and files, and deletes them.

We now discuss the catalog service and the metadata store in detail, followed by the two key features: extended MoR (Merge-on-Read) and multi-table transactions.

3.1 Catalog Service

The catalog service provides a high-level gRPC [32] API, a high-performance RPC (Remote Procedure Call) framework developed by Google. Compute engines use this gRPC API to access and modify metadata, with multiple server instances deployable for scalability. Many functions, such as *getTable()* and *createTable()*, are CRUD operations implemented using ERQL, while *execQuery()* allows clients to directly submit a limited set of ERQL queries for data-intensive operations (e.g., involving `FILELIST`).

Inside the catalog service, a transpiler translates ERQL commands into the backend metadata store's native query language (SQL for our PostgreSQL prototype). Because ERQL is a high-level declarative language, there is no restriction on the choice of metadata store, and transpilers for other data systems may be added as plugin modules. In the ERQL-to-SQL translation scheme, each entity or relationship type maps to a table, and read queries translate into inner joins on object IDs. For abstract entity types, queries expand into a union of subqueries over derived concrete entity types. Because union scales optimization complexity linearly with subquery count (unlike a join), this avoids excessive query planning overhead.

Because the volume of file-level metadata remains substantial even after offloading statistics to OTFs, far exceeding other metadata types, we store it in a partitioned table grouped by `FILELIST` history. Each partition stores file-level metadata with two additional fields, *start_vid* and *end_vid*, which determine the file's local lifetime within that `FILELIST` history. Any query against `FILELIST` translates into a query against the corresponding table partition with predicates on *start_vid* and *end_vid*. For each file, the metadata store also maintains a global `FILELIST` count, tracking the number of `FILELIST` histories in which the file is currently alive.

Lastly, second-order metadata, including entity and relationship type definitions and retention policies, is stored as catalog tables in the metadata store and cached in memory by each catalog service server.

3.2 Metadata Store

There is a wide range of OLTP (Online Transaction Processing) systems, including RDBMSs [40, 43, 52, 73], graph databases [4, 34, 46, 75, 80], column-family stores [30, 69, 70], document stores [7, 44], and key-value stores [3, 41, 79]. The primary factor in selecting the

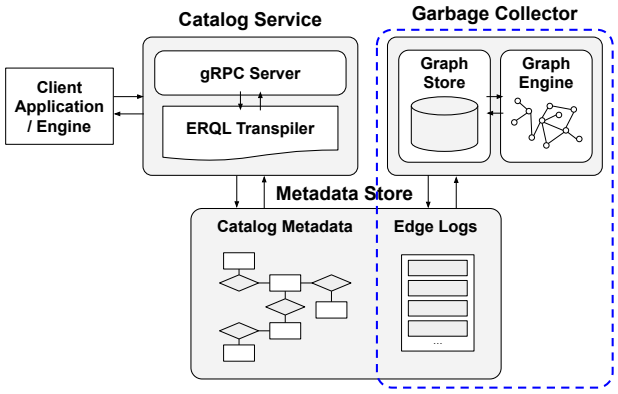

Figure 4: The high-level architecture of Raven.

underlying system was robust support for multi-row transactions with a strong isolation level, without which advanced features like multi-table transactions and garbage collection are difficult to implement correctly. Although we currently use PostgreSQL for its maturity and ease of development, other OLTP systems providing serializable transactions are viable alternatives.

While scalability is a valid concern, offloading file-level statistics to OTFs and adopting a constrained relational view alleviate pressure on the metadata store. In production, the compute engine or catalog can further reduce stress on the store by caching dataset snapshots and applying incremental updates via `FILELIST DELTA` commands. Consequently, for a single enterprise, large-scale distributed systems like FoundationDB [79] or Spanner [19] would be required only in extreme circumstances. Fault tolerance is another important concern, which can be addressed using standard replication schemes or more specialized solutions, such as Amazon Aurora [76] and Neon [47].

3.3 Base Transaction Protocol

Before introducing the two features addressing OTF limitations, we describe Raven's core table-level concurrency control mechanism, which combines OCC (Optimistic Concurrency Control) with MVCC (Multi-Version Concurrency Control). A typical read-write path proceeds as follows:

(1) **Snapshot Retrieval:** The client retrieves the latest table snapshot and its version ID from Raven. The table snapshot is retrieved from the metadata store in a single transaction.
(2) **Read Phase:** Via ERQL queries to Raven, the client retrieves the list of metadata and/or data files visible in the current table snapshot. The client reads the target files.
(3) **Write Phase:** The client writes new data and/or metadata files and sends the list of newly created or logically deleted files to Raven, which incrementally builds an ERQL command to apply these changes.
(4) **Raven Commit:** Upon invoking *commit()*, the catalog service translates the ERQL command into SQL and executes it in the metadata store as a single serializable transaction, acting as a CAS operation on the target table snapshot. The new files become visible to other clients only after the transaction commits. If the target table snapshot has a higher version ID than what was initially retrieved, the transaction is aborted.
(5) **Post-Commit:** If *commit()* aborts due to a transaction failure in the metadata store, the client handles recovery (e.g., retrying or deleting written files).

Throughout the remainder of this paper, an OTF commit under Raven refers specifically to generating the relevant OTF files (delta log and checkpoint files for Delta Lake; manifest, manifest list, and metadata files for Iceberg). Thus, Iceberg omits its final catalog-level CAS operation, whereas Delta Lake still requires an atomic rename to create a new delta log file.

3.4 Extended Merge-on-Read

We next introduce extended MoR (Merge-on-Read) to reduce commit latency and metadata bloat. Extending the data-layer MoR strategy for small updates [50] to the metadata layer, extended MoR targets small, frequent updates where OTF commit latency and metadata file generation costs cannot be amortized as in bulk updates.

Because Raven already tracks file metadata in its transactional metadata store, it can defer OTF commits rather than eagerly generating OTF metadata files on every update. In the fast write path, the client follows the base OCC protocol to simply register new data files in Raven with a "new data" tag. Because these files are not yet tracked by the OTF, the read path merges them during scans. In the read path, after retrieving the table snapshot, the client issues ERQL queries to fetch both the OTF metadata files (the metadata file for Iceberg; the latest delta log file for Delta Lake) and all "new data" files visible in the current table snapshot. The client then reconstructs the OTF snapshot and merges its file list with the list of "new data" files.

To prevent the list of files subject to merge from growing indefinitely, periodic **metadata compaction** materializes "new data" files into OTF metadata, following the base OCC protocol. During the write phase, all "new data" files visible in the current snapshot are committed to the respective OTF. The resulting OTF files and tag updates (from "new data" to "data") for the target data files are then committed to Raven, making the changes visible. If the Raven commit aborts due to a conflict, the new OTF files are reclaimed safely as they are not visible until the final Raven commit succeeds. Performing OTF commits only periodically yields (1) fewer small metadata files and (2) lower average commit latency. Unlike batching, extended MoR preserves data freshness because Raven commits become visible prior to OTF materialization. However, because the metadata store lacks file-level statistics, "new data" files cannot be pruned before scanning even if they contain no relevant data, potentially impacting read performance. Currently, this feature supports only fast appends, but it is straightforward to extend it to updates and deletes.

3.5 Multi-Table Transactions

Raven also extends the base protocol to support multi-table OTF transactions under snapshot isolation. Like the base transaction protocol, all clients must go through Raven to avoid observing uncommitted changes.

(1) **Snapshot Retrieval:** The client invokes *startTransaction()* with target table names (both read and write tables) to retrieve their snapshots and version IDs from the metadata store in a single transaction.
(2) **Read Phase:** The client fetches the OTF metadata files (Iceberg metadata file; Delta Lake latest delta log) visible in the current table snapshots from Raven via ERQL queries, reconstructs the OTF snapshots, and scans the data files.
(3) **Write Phase:** The client writes new files and sends their paths to Raven, which constructs an ERQL command.
(4) **OTF Commit:** The client performs OTF commits, captures the newly generated OTF files, and sends them to Raven to append to the ERQL command.
(5) **Raven Commit:** The catalog service translates the ERQL command into SQL and submits it as a single serializable transaction to the metadata store, acting as a multi-row CAS across all modified table snapshots. New data and OTF files become visible only after this commit succeeds.
(6) **Post-Commit:** On abort, written data and OTF files are deleted. This cleanup is safe because uncommitted files are never visible to clients going through Raven.

Committing metadata changes as a single serializable transaction guarantees a consistent cross-table snapshot. However, because Raven only applies the final catalog transaction to modified tables, it only detects ww-dependencies and permits write skew. If any OTF commit fails (e.g., during Delta Lake's atomic rename), the entire transaction aborts and deletes all newly created files. While the protocol does not currently handle orphan files from client crashes, a future remedy is a pre-commit step that allocates file names in a session-referenced temporary file list before file creation: moving them to active table file lists upon commit, or reclaiming them after the session ends. For Delta Lake, this mechanism is essential to prevent orphaned delta log files from blocking subsequent client commits.

4 Garbage Collection

Garbage collection is one of the most critical maintenance operations in a data lakehouse. Because data modifications continuously generate new files, older files and their associated metadata eventually become obsolete and must be reclaimed. Without strict safety guarantees, a garbage collector can delete active data, compromising data integrity. We first diagnose the limitations of garbage collection in existing OTFs.

4.1 OTF Limitations

OTFs, including Delta Lake, Iceberg, and Hudi, primarily rely on retention-based garbage collection policies [6, 23], removing older table snapshots and their underlying data files once they exceed a configured time threshold. However, this compromises safety whenever older snapshots remain in active use. For instance, if an ETL pipeline accessing a snapshot runs longer than the retention window, the garbage collector may prematurely delete the snapshot's files, causing the job to fail. Furthermore, existing OTFs fail to account for access paths that span multiple tables or assets. For example, Delta Lake's `VACUUM` command reclaims physical files without checking for incoming references from shallow clones, risking corruption of dependent datasets [23]. Iceberg's `expireSnapshots()` procedure maintains local safety by computing the set difference between expired and active snapshots to identify unreferenced files. However, this safety guarantee relies on a restrictive constraint: cloning is confined to the scope of a single table directory path, precluding support for cross-table shallow clones.

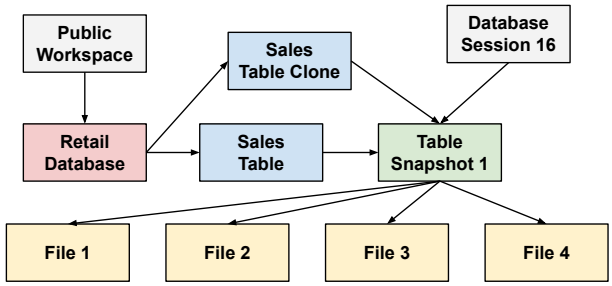

Figure 5: Graphical representation of example access paths, consisting of references between different entities.

Fundamentally, OTFs fail to track references across multiple file access paths. As illustrated in Figure 5, data files can be referenced through more than one distinct path:

- A workspace references a database, which references a table, which points to a snapshot that directly references data files.
- A cloned table references a snapshot in a separate base table, sharing the underlying data files.
- An active database session holds a reference to the table snapshot it is currently accessing.

Implementing precise reference tracking across these paths directly on top of OTFs is challenging because metadata is dispersed across multiple files in a cloud object store. Traversing these reference paths incurs substantial overhead due to high cloud object store latency. Furthermore, weak transactional guarantees make even straightforward mechanisms, such as reference counting, difficult to implement. Finally, the unstructured nature of native cloud storage APIs (key-value abstraction) further compounds these challenges.

4.2 Problem Formalization

To address the limitations of existing solutions from first principles, we formalize garbage collection over three object operations: *create*, *access*, and *delete*. Given an input schedule s of *create* and *access* operations, the garbage collector must output a *delete* schedule g such that the combined schedule $s \oplus g$ is both (1) **complete**, where every object eventually has a *delete* operation after its last *access*, and (2) **safe**, where each object's *delete* succeeds its *create* and all *access* operations so no premature deletions occur.

Because garbage collection operates in real time, it requires a **liveness criterion** to determine whether an object will be accessed in the future. Existing OTFs rely on a **retention period** policy that assumes objects older than a threshold are obsolete, a heuristic that can fail and cause premature deletions. An alternative is **reachability**: modeling objects and their references as a directed graph, an object is considered alive only if a valid access path exists from

a designated root set, allowing unreachable objects to be safely reclaimed. While OTFs fail precisely because they do not fully account for reachability, reachability alone is also insufficient in data lakehouses because accidental user operations (e.g., dropping a table) can prematurely delete references. Consequently, combining both policies is necessary to ensure safety.

4.3 Referential Relationships

To support garbage collection, we extend ERQL with **referential relationships** that natively incorporate reachability semantics into the metadata model (Figure 5). Because not every relationship is a reference, users must specify via DDL whether a relationship type is also a reference, as follows:

```
ALTER RELATIONSHIP Contains(Database, Asset)
SET Database REFERENCES Asset;
```

For an access path to be considered valid, it must start from an entity belonging to a predefined **root set**. We call the corresponding entity types **root entity types**, which can be declared as shown below:

```
ALTER ENTITY Workspace SET ROOT;
ALTER ENTITY Session SET ROOT;
```

To support retention-based recovery as mentioned above, we also allow users to configure a retention period for references. When a reference is dropped, it is marked rather than deleted for the specified duration. An entity reachable only via a path containing such a "ghost" reference is no longer visible, yet remains recoverable during its retention window and is deleted only after the retention period expires. For example, the default retention period for `Contains(Database, Table)` can be set as follows:

```
ALTER RELATIONSHIP Contains(Database, Table)
SET RETENTION INTERVAL '7 days';
```

The default retention period can be overridden with the `OVERRIDING` keyword at the time a reference is dropped.

Raven defines an entity as alive if it is reachable from a root entity via a path of references, each of which is either (1) active or (2) deleted but unexpired. This constraint cannot be enforced by Raven alone; for example, once a compute engine retrieves the list of data files for a table snapshot, Raven cannot stop the engine from reading them after the last reference is dropped (e.g., by a `VACUUM` operation). Instead, Raven provides an interface for clients to cooperate in enforcing this constraint. To safely access an asset snapshot, the client creates a root *session entity* referencing the target snapshot so that it is not deleted even if other references are dropped. Abandoned session entities where a client fails to delete its session can be mitigated using leases with a Time to Live (TTL), reinforced by a heartbeat mechanism; cooperative clients locally track their lease and abort execution if it expires. Because clients in this architecture are internal execution engines and applications within the same enterprise lakehouse, we do not consider non-cooperative or adversarial behavior to be a realistic assumption.

Because file-level metadata is represented using `FILELIST`, file-level references are implicit within `FILELIST` metadata. A file can span multiple `FILELIST` histories, so its references are the union of all references across those `FILELIST` histories. Accordingly, the metadata store maintains a global `FILELIST` count for each file, representing how many `FILELIST` histories (not `FILELIST` versions) in which the file is currently alive.

We model the referential structure as a directed graph $G = (V, E)$, where V is the set of entities and E contains active or deleted, but unexpired, referential relationships. Given a root set R, the garbage collector identifies any vertex $v \in V$ unreachable from all $r \in R$. Because Raven uses reference counting, which cannot handle cycles, it prohibits DDL declarations that introduce referential cycles into the ER schema. This imposes minimal constraint in practice, as lakehouse referential structures are naturally acyclic, and non-referential relationship schemas remain unrestricted.

4.4 Implementation

4.4.1 Overview. Because the metadata store maintains the complete referential graph topology, including file-level references, transactionally under serializable isolation, it serves as a reliable source of truth for consistent garbage collection snapshots. However, executing graph traversals directly against the metadata store is problematic for two key reasons. First, graph algorithms inherently exhibit poor data locality due to irregular structural topologies and random access patterns [28, 39]. Second, garbage collection traversals require only graph connectivity and minimal algorithm-specific state (e.g., a mark bit per vertex). Consequently, direct traversal against the metadata store causes severe read amplification and random disk I/O. Furthermore, running these heavy traversal queries directly on the metadata store risks resource contention, potentially disrupting concurrent user operations.

Based on these considerations, we design the garbage collector as a specialized graph analytics engine that maintains a **dynamic graphical view** of the catalog metadata. The garbage collector periodically ingests updates from the metadata store, analyzes the graph to identify unreachable objects, and deletes them. The garbage collector encompasses the following three components (Figure 4):

(1) **Metadata Store**: The metadata store is the single source of truth that stores the underlying metadata. It maintains a log of updates retrieved by the garbage collector.
(2) **Graph Store**: The graph store maintains an on-disk checkpoint of the graph state used for recovery after any failure.
(3) **Graph Engine**: The graph engine maintains an in-memory graph state, on which the garbage collector operates to identify unreachable objects. The in-memory graph can handle random access patterns without performance issues.

The graph store and graph engine are co-located on a dedicated machine, while the metadata store operates separately.

4.4.2 Main Process. The metadata store logs changes to the object graph, additions and deletions of referential relationships, across two tables. Each log record is identified by source and destination object IDs (and version IDs for `FILELIST`). Time is divided into epochs, with one garbage collection cycle per epoch. The main process initiates a cycle by incrementing the epoch ID, retrieving log records up to the previous epoch (all additions plus deletions whose retention expired), and updating the graph store. The garbage collector only requires knowing if an edge count is non-zero at an epoch boundary, so the graph store tracks only total addition and deletion counts per edge ID (and the epoch ID for durability).

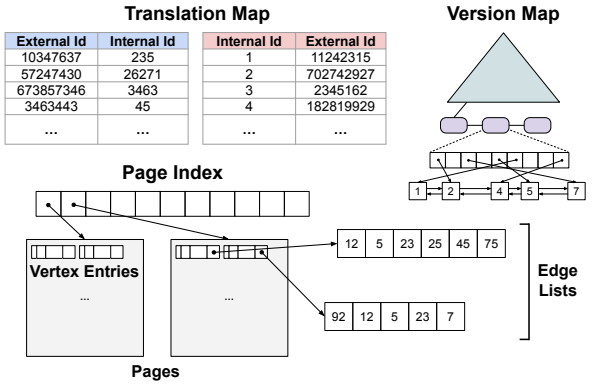

Figure 6: Main data structures of the graph engine.

There may be multiple updates to the same edge, so the counts are aggregated in memory before being atomically applied to the graph store. After changes are committed, the corresponding log records in the metadata store are removed in a single transaction.

The in-memory graph engine receives the same updates as the graph store: additions first, followed by deletions. Since the system only requires correct graph state at epoch boundaries, updates can be reordered as long as an addition precedes its corresponding deletion, guaranteed by the update scheme. Figure 6 illustrates the graph engine's main data structures. The **translation map** maintains a mapping between external object IDs and internal vertex IDs. The **page index** points to dynamically allocated pages of vertex records, where each record tracks a reference count, edge list pointer, and metadata. Edge lists use dynamic arrays when small and hash tables otherwise. The engine also maintains a free list of pages whose load factor is lower than a threshold. Lastly, the **version map** maps a `FILELIST` object ID to its version chain, which keeps track of the list of live versions/snapshots.

Unlike tracing mechanisms, which require full graph traversals from the root set, the graph engine uses reference counting to perform a selective traversal starting from unreachable objects identified during updates. Starting with this initial active set, the garbage collector performs a traversal and in-place updates (Algorithm 1) to locate unreachable objects and deallocate their in-memory vertex records. It then deletes the corresponding entities and relationships from the metadata store. Once these changes to the metadata store are committed, unreachable edge entries are deleted from the graph store to limit its size. Unlike references dropped by clients that are logged in the metadata store, references that become unreachable as a byproduct are deleted directly by the garbage collector, reducing data transfer overhead. The reachability of a root entity itself is tracked by an incoming reference from a virtual super root entity.

A file is visible in a `FILELIST` version if its version ID falls between *start_vid* and *end_vid*. To track these implicit file references without storing explicit relationships, the graph engine maintains a **version chain**, a hash table index and an ordered linked list of live version IDs, for each `FILELIST` history. When a `FILELIST` version is deleted, its entry in the chain is removed, and the global `FILELIST` count of the files whose local lifetimes fall between neighboring versions is decremented. Background threads scan the metadata store for files with a global `FILELIST` count of zero and delete them. By tracking files at `FILELIST` granularity, rather than individually, the graph engine avoids managing billions of file-level metadata objects, enabling the entire graph structure (expecting tens of millions of other catalog objects at most) to fit in the memory of a single machine.

Algorithm 1 Garbage Collection Algorithm

```
1:  D ← ∅
2:  while A is not empty
3:      v ← an element from A
4:      A ← A \ {v}
5:      D ← D ∪ {v}
6:      for (v, w) ∈ v.outgoing_edges
7:          w.ref_count ← w.ref_count − 1
8:          if w.ref_count = 0
9:              A ← A ∪ {w}
10:     // update reference graph
11:     V ← V \ {v}
12:     E ← E \ v.outgoing_edges
13: return D
```

4.4.3 Durability. To ensure fault tolerance, the graph store maintains a checkpoint of the graph state in RocksDB [41], a high-performance embedded key-value store, across two collections: edge additions and edge deletions. Because edges can be updated repeatedly, each collection maps edge IDs to the epoch ID of the latest operation and their total operation count. Because metadata store logs are deleted only after the graph store update commits, the graph state can be reconstructed from the metadata store even if a crash occurs during updates. For idempotency, log aggregates (grouped by epoch ID) are applied to the respective graph store collection only if their epoch ID exceeds the edge's latest recorded epoch ID during recovery. The graph engine is then rebuilt from the graph store before resuming garbage collection. Although the garbage collector may reattempt deletions performed before the failure, deletion operations on metadata and files are idempotent. To prevent double-decrementing the global `FILELIST` count, the decrement is computed strictly from the files actually removed from the relevant file lists in a single transaction.

5 Experimental Evaluation

We conduct our experiments across four physical machines on CloudLab (Wisconsin) [18], each equipped with a 10-core Intel Xeon Silver 4114 and 192 GiB of DDR4 memory interconnected via 25 Gbps Ethernet, using AWS S3 [5] in US East (Ohio) for primary data storage. The four machines respectively host: (1) the client application (Raven's gRPC client paired with either Iceberg 1.10 and DuckDB 1.4.4, or Delta Lake Kernel 4.0 and its default engine); (2) the Raven catalog service; (3) the Raven metadata store; and (4) the Raven garbage collector. For comparative evaluations against DuckLake, we deploy (1) DuckDB 1.4.4 paired with DuckLake 0.3 client; and (2) DuckLake backend (PostgreSQL).

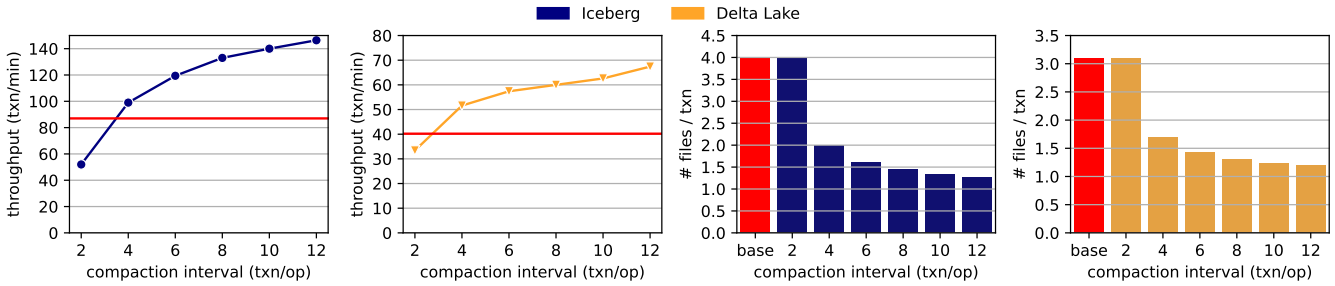

Figure 7: Throughput and total number of files created per transaction for the extended MoR implementation of Iceberg and Delta Lake across different compaction intervals. The baseline results without Raven integration are colored in red.

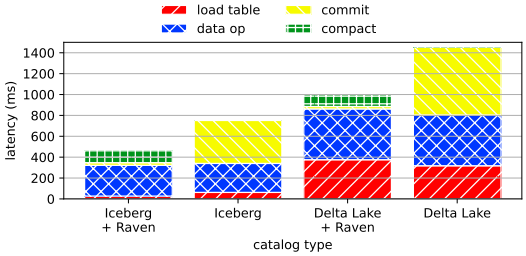

Figure 8: Mean latency breakdown of a single insert with and without extended MoR (compaction interval of 10 txn/op). Raven integration has lower combined commit and compact time than the commit time of OTF alone.

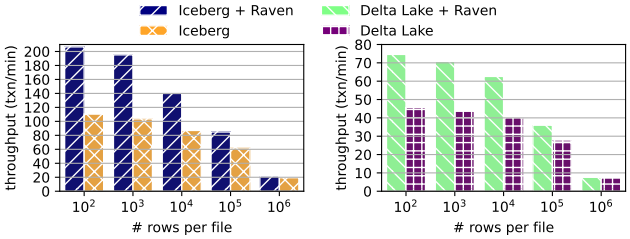

Figure 9: Insert transaction throughput for varying row counts per data file. Approximate file sizes are 14 KB, 96 KB, 0.9 MB, 9.2 MB, and 92 MB, respectively.

5.1 Extended Merge-on-Read

We evaluate how extended MoR reduces metadata bloat and commit latencies for Iceberg and Delta Lake. We use a microbenchmark where a single thread inserts data of a fixed size into the *Store Sales* table of the TPC-DS benchmark.

5.1.1 Compaction Interval. We first measure overall throughput and the number of files created while varying the compaction interval (the number of transactions per OTF commit) for both Iceberg and Delta Lake integrations (Figure 7). Compaction operation itself is a single transaction, so the compaction interval of k results in a compaction per $k-1$ inserts. Hence, with a compaction interval of 2, throughput is lower than the non-Raven baseline because metadata files are created and committed to Raven for every insert. However, as the interval increases, throughput improves significantly by amortizing metadata creation overhead. For example, at an interval of 10, Iceberg throughput reaches 140 txn/min, 61% higher than the baseline (87 txn/min), while Delta Lake reaches 62 txn/min, 55% higher than the baseline (40 txn/min). We also observe a substantial reduction in files created per transaction as the interval increases (lower metadata bloat). At an interval of 10, the number of files created per operation drops to 1.33 for Iceberg and 1.23 for Delta Lake, well below their respective baselines (4 and 3.1). As shown in Figure 8, the mean latency breakdown at an interval of 10 reveals that a lower amortized commit time is the primary contributing factor, as Raven's combined commit and compact latency is much lower than the commit latency of the baseline Iceberg and Delta Lake.

5.1.2 Data File Size. We next evaluate throughput as we increase the number of rows per data file by orders of magnitude. The results are shown in Figure 9. As expected, the throughput gap between the baseline OTFs and the Raven integrations narrows as data operation costs become dominant. However, even for data files with 100,000 rows (approximately 9.2 MB), Raven improves throughput by 36.8% and 31.4% for Iceberg and Delta Lake, respectively.

5.1.3 Read Performance. Finally, we show that extended MoR has minimal impact on Iceberg read performance. We populate tables with and without extended MoR (1,008 data files, 100,000 rows per file ≈9.2 MB, and a compaction interval of 10) and evaluate read queries across fixed selectivities (Figure 10). We deliberately chose the number of data files so that 7 files remain untracked by the OTF. For Iceberg, scan parallelism chosen by DuckDB during planning strongly influences runtime, as seen in the execution time drop from 2% to 4% selectivity. Latency differences between the two configurations remain small across selectivities, showing that the engine's parallel scan easily absorbs the overhead of scanning a few extra data files. For Delta Lake, read performance is governed by the Delta Kernel implementation rather than data file counts. Specifically, the Delta Kernel lazily fetches and parses log files from S3 during snapshot reconstruction, adding substantial latency. Variations in post-checkpoint log file counts mask the effect of scanning extra data files, making Delta Lake results inconclusive.

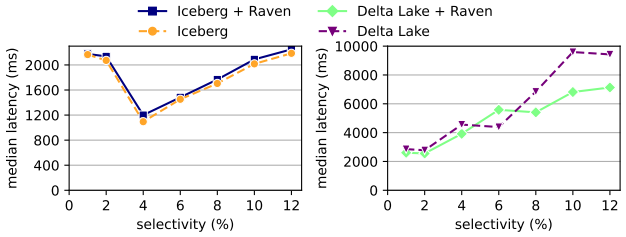

Figure 10: Median scan latency across varying selectivities with and without extended MoR.

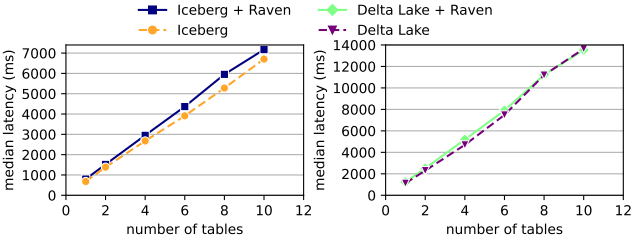

Figure 11: Median latency of a multi-table insert with and without a multi-table transaction across varying table counts.

5.2 Multi-Table Transactions

5.2.1 OTF Integration. Next, we evaluate the overhead incurred by Raven's multi-table transactions compared to sequential single-table transactions in baseline OTFs. Using 24 tables generated from the TPC-DS schema, our microbenchmark randomly selects a subset of tables and inserts 10,000 rows into each (Figure 11). For both Delta Lake and Iceberg, multi-table transactions introduce overhead from the per-table catalog RPCs required to fetch OTF metadata file paths. The mean latency breakdown for six-table transactions (Figure 12) reveals that Iceberg's overhead stems from both table loading and commits (176 ms and 279 ms across target tables, respectively), as each table is also refreshed during commit. On the other hand, Delta Lake's overhead stems primarily from table loading (366 ms) rather than commits (133 ms), which do not require a table refresh. Batching these catalog RPCs is a potential optimization for future work.

5.2.2 DuckLake Comparison. We next compare Raven's catalog transaction path for multi-table transactions against DuckLake. To isolate the throughput and contention behavior of the catalog concurrency control mechanism, we minimize data overhead by omitting OTF integration and using lightweight single-column tables. Specifically, each transaction inserts a single row into 1 to 5 tables selected uniformly at random from a pool of 1,000 tables. As shown in Figure 13, Raven's throughput scales from 23.6 txn/s at 10 threads to 146.4 txn/s at 90 threads. In contrast, DuckLake reaches a maximum of 25.4 txn/s at 40 threads. Raven enforces table-level OCC, causing conflicts only when the target tables overlap. On the other hand, DuckLake serializes global catalog snapshot creation, creating a central bottleneck regardless of actual data conflicts. Retry validation logic prevents redundant execution when

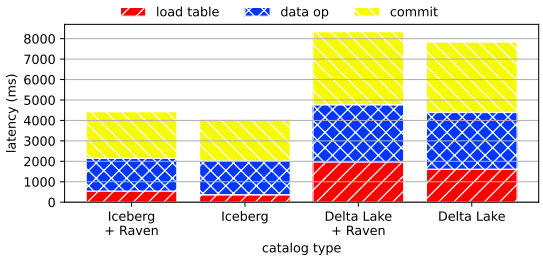

Figure 12: Mean latency breakdown of a multi-table insert into 6 tables with and without a multi-table transaction.

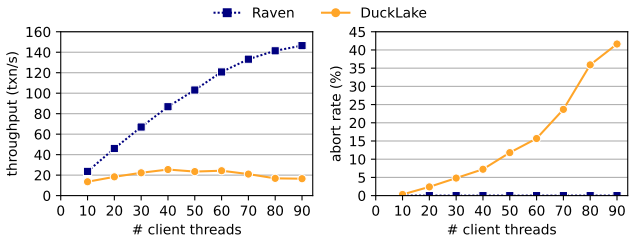

Figure 13: Throughput and abort rate of multi-table inserts across varying numbers of client threads. DuckLake's snapshot-based OCC causes higher contention.

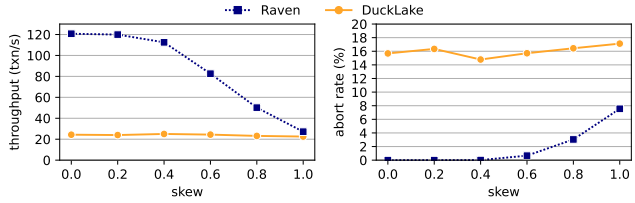

Figure 14: Throughput and abort rate of multi-table inserts across varying Zipfian skew factors (60 threads). Raven's performance converges to DuckLake's due to higher contention.

failures stem solely from catalog snapshot creation. But, failed snapshot creation still causes multiple retries and eventually leads to a transaction abort after 10 retries. As a result, DuckLake's abort rate reaches 41.6% at 90 threads. For Zipfian distribution, DuckLake's throughput remains flat, whereas Raven's throughput drops toward DuckLake's baseline with increasing skew factor due to higher table-level contention (Figure 14).

5.3 Statistics Overhead

We show that Raven (with Iceberg) suffers substantially less statistics overhead than DuckLake using a single table of 100,000 single-row files across varying column counts.

5.3.1 Metadata Footprint. We measure PostgreSQL size to evaluate the catalog metadata footprint. As shown in Figure 15, DuckLake's footprint scales linearly with column count, reaching 3.27 GB at 320 columns due to file-level statistics. In contrast, Raven's catalog

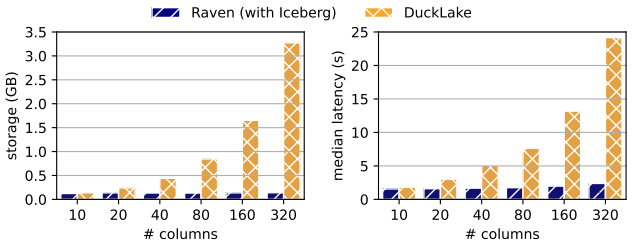

Figure 15: Catalog metadata footprint and single-row query latency across varying column counts. DuckLake suffers from increasing overhead due to file-level statistics.

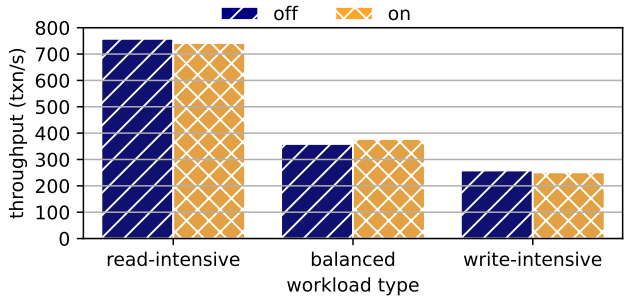

Figure 16: Throughput of different workloads with and without garbage collection, showing its minimal impact.

storage size remains stable below 140 MB as it offloads file-level statistics to Iceberg.

5.3.2 Read Performance. We measure query overhead by randomly selecting a single row from the table. Raven benefits from Iceberg, which can prune unnecessary manifest files and parallelize metadata loading, limiting its median latency increase from 1.82 s to 2.38 s in contrast to DuckLake's increase from 1.55 s to 24.17 s as column count increases from 10 to 320 (Figure 15). DuckLake retrieves file-level statistics from PostgreSQL using a full table scan with predicate filters. Because the physical volume of statistics scanned is proportional to the column count, DuckLake faces scalability issues.

5.4 Garbage Collection

Lastly, we conduct an evaluation of the garbage collector.

5.4.1 Overhead. We first demonstrate that garbage collection imposes minimal overhead on concurrent metadata operations. Across a database of 1,000 tables, 60 client threads execute three different types of operations: the *refresh* operation is a read-only operation that scans new data files that have been added since the last operation; the *insert* operation inserts new data into given tables; and the *compact* operation (different from compaction in extended MoR) merges multiple new data files into a single data file and invokes *expireSnapshots()*, retaining only the 10 most recent snapshots and triggering the garbage collector. To isolate GC overhead from transactional contention, each thread operates exclusively on its own

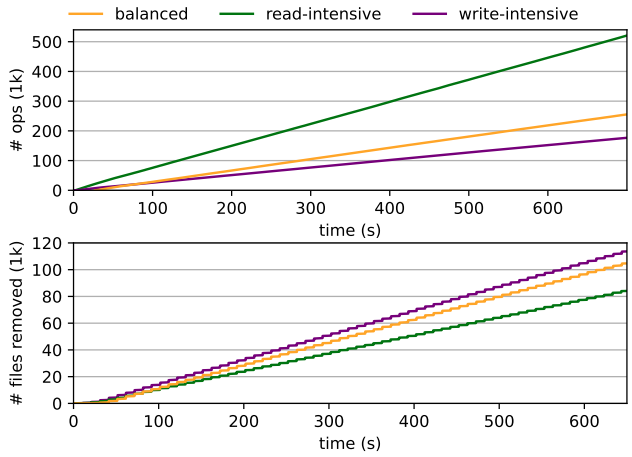

Figure 17: Total number of client operations and number of files removed as a function of time.

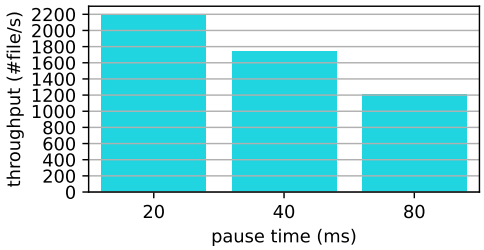

Figure 18: Rate of file removal for different client pause times.

assigned subset of tables. We evaluate three workloads with GC enabled and disabled: read-intensive (80% read, 20% write), balanced (50% read, 50% write), and write-intensive (20% read, 80% write), using a 10-second epoch period. Figure 16 illustrates the main result: overall performance shows negligible difference regardless of GC activity. Figure 17 shows operation throughput and file deletions over time. Metadata operation rates remain steady, while file deletions follow a "jagged" pattern due to batch removal at the end of each epoch.

5.4.2 Scalability. Finally, we evaluate the scalability of the garbage collector. We first populate 1,000 tables with 300,000 data files using a mix of insert and compact operations. Next, 30 client threads select tables from their assigned groups and expire 10 snapshots per operation until there are no snapshots left to expire. In this setup, the GC workload generated per operation far exceeds client processing time. Consequently, executing snapshot expirations in a tight loop would exceed S3's rate limit (3,500 DELETEs/s per partitioned prefix), triggering throttling. We therefore regulate client throughput by introducing a pause interval between consecutive operations. Figure 18 reports file deletion rates across three pause times (20 ms, 40 ms, and 80 ms). At a 20 ms pause, the lowest interval before hitting S3 limits, the system deletes 2,183.6 files/s. To verify that GC throughput is mainly constrained by S3 API limits and overheads,

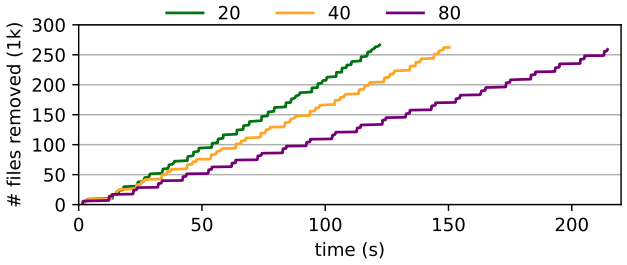

Figure 19: Total number of files removed by the garbage collector over time for different client pause times.

we repeated the experiment on a local filesystem with zero pause time, reaching 3,256.8 files/s. Lastly, Figure 19 shows cumulative file deletions over time, following a near-linear trajectory with a "jagged" pattern caused by epoch-based bulk removals.

6 Related Work

Prior work has also addressed multi-table transactions in data lakehouse systems. Nessie [53] uses Git-like branching semantics to isolate multi-table modifications before atomically merging them into the main branch. DuckLake [26] employs a client-side OCC mechanism that optimistically generates a global catalog snapshot, executing conflict validation if the RDBMS transaction fails. LakeVilla [31] provides a pluggable module for Iceberg and Delta Lake, in which the client places marker files to lock table snapshots and track cross-table dependencies and commits to a global version log for cross-table consistency. TreeCat [49] supports multi-table transactions based on fine-grained optimistic predicate locking. Finally, Databricks announced support for multi-table transactions, but it remains restricted to proprietary compute engines integrated with Unity Catalog as of September 2026 [22].

The garbage collection problem persists across many different domains. In programming languages, automatic heap management [35] relies primarily on global tracing (e.g., mark-sweep) or reference counting, employing advanced techniques, including concurrent [13, 25, 60, 78] and generational garbage collection, and deferred [24] and cyclic reference counting [12]. The MVCC mechanism of database systems requires garbage collection to remove tuple versions no longer visible to any active transactions, performed in the background [72] or as part of query processing [14, 37]. Partitioning schemes that allow efficient bulk deletion have also been devised [36, 48].

7 Conclusion

We addressed the core architectural dilemma of data lakehouse metadata management. While file-based OTFs scale analytical workloads, they suffer from metadata bloat, high commit latencies, absence of multi-table transactions, and unsafe garbage collection. Database-only catalogs bottleneck on file-level statistics. To bridge this divide, we presented Raven, a catalog whose hybrid design isolates transactional control metadata from analytical metadata. Raven enables extended MoR to curb metadata bloat, coordinates multi-table transactions, and models asset lifecycles via an extensible ER abstraction for safe garbage collection. Our evaluation confirms that Raven lowers commit latency and metadata growth for Apache Iceberg and Delta Lake, maintains catalog storage orders of magnitude smaller than DuckLake, and deletes obsolete files at scale with minimal disruption.

Looking ahead, several important challenges remain. Given the open architecture of data lakehouses, **access control** is harder to address than in closed systems such as data warehouses. Although enterprise solutions such as Unity Catalog exist [21], enforcing fine-grained policies such as row-level filters across different engines remains difficult. Second, we have not evaluated Raven at **an extreme scale**, which would likely require a metadata store backed by a highly scalable transactional system. Finally, **optimizing the mapping** from the ER model to the underlying storage remains an open opportunity to further improve performance by appropriately selecting a table representation for the metadata.

Acknowledgments

This research was supported by Dolby Laboratories.

AI-Generated Content Acknowledgment

The authors used Google Gemini to improve the grammatical fluency and readability of the manuscript. The authors also used GitHub Copilot and Google Gemini for coding and script development assistance.

References

[1] D. Abadi. The file explosion problem in Apache Iceberg and what to do when it happens to you. https://www.starburst.io/blog/apache-iceberg-files/, 2025. Accessed: October 15, 2026.

[2] D. Abadi and A. Mott. Data Virtualization will Become a Core Component of Data Lakehouses in 2024. https://vmblog.com/prediction/starburst-2024-predictions-data-virtualization-will-become-a-core-component-of-data-lakehouses-in-2024/, 2024. Accessed: October 15, 2026.

[3] Amazon Web Services. Amazon DynamoDB. https://aws.amazon.com/dynamodb/, 2026. Accessed: October 15, 2026.

[4] Amazon Web Services. Amazon Neptune. https://aws.amazon.com/neptune/, 2026. Accessed: October 15, 2026.

[5] Amazon Web Services. Amazon S3. https://aws.amazon.com/products/storage/, 2026. Accessed: October 15, 2026.

[6] Apache Iceberg. Branching and Tagging. https://iceberg.apache.org/docs/latest/branching/, 2026. Accessed: October 15, 2026.

[7] ArangoDB. ArangoDB. https://arangodb.com/, 2026. Accessed: October 15, 2026.

[8] M. Armbrust, T. Das, L. Sun, B. Yavuz, S. Zhu, M. Murthy, J. Torres, H. van Hovell, A. Ionescu, A. Łuszczak, M. Świtakowski, M. Szafrański, X. Li, T. Ueshin, M. Mokhtar, P. Boncz, A. Ghodsi, S. Paranjpye, P. Senster, R. Xin, and M. Zaharia. Delta lake: high-performance ACID table storage over cloud object stores. *Proc. VLDB Endow.*, 13(12):3411–3424, Aug. 2020.

[9] M. Armbrust, A. Ghodsi, R. Xin, M. Zaharia, et al. Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. In *The Conference on Innovative Data Systems Research, CIDR*, 2021.

[10] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, SIGMOD '15, page 1383–1394, New York, NY, USA, 2015. Association for Computing Machinery.

[11] N. Armenatzoglou, S. Basu, N. Bhanoori, M. Cai, N. Chainani, K. Chinta, V. Govindaraju, T. J. Green, M. Gupta, S. Hillig, E. Hotinger, Y. Leshinksy, J. Liang, M. McCreedy, F. Nagel, I. Pandis, P. Parchas, R. Pathak, O. Polychroniou, F. Rahman, G. Saxena, G. Soundararajan, S. Subramanian, and D. Terry. Amazon Redshift Re-invented. In *Proceedings of the 2022 International Conference on Management of Data*, SIGMOD '22, page 2205–2217, New York, NY, USA, 2022. Association for Computing Machinery.

[12] D. F. Bacon and V. T. Rajan. Concurrent cycle collection in reference counted systems. In *European Conference on Object-Oriented Programming*, pages 207–235. Springer, 2001.

[13] H. G. Baker. The treadmill: real-time garbage collection without motion sickness. *SIGPLAN Not.*, 27(3):66–70, Mar. 1992.

[14] J. Böttcher, V. Leis, T. Neumann, and A. Kemper. Scalable garbage collection for in-memory MVCC systems. *Proc. VLDB Endow.*, 13(2):128–141, Oct. 2019.

[15] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas. Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering*, 38(4), 2015.

[16] R. Chandra, H. Chen, R. Matharu, S. Cai, J. Chen, P. Dutta, B. Ghita, T. Greenstein, G. Holla, P. Huang, Y. Huo, A. Ionescu, A. Ispas, T. Januschowski, V. Karajgaonkar, S. Leone, D. Lewis, A. Li, N. Li, C. Lian, S. Link, Q. Lu, Y. Ma, C. Pettitt, V. Prabhakaran, B. Raducanu, K. Rong, P. Roome, S. Shetty, S. Smith, X. Sun, Y. Tang, W. Wen, L. Xia, J. Zeng, B. Zhang, R. Xin, and M. Zaharia. Unity Catalog: Open and Universal Governance for the Lakehouse and Beyond. In *Companion of the 2025 International Conference on Management of Data*, SIGMOD/PODS '25, page 310–322, New York, NY, USA, 2025. Association for Computing Machinery.

[17] P. P.-S. Chen. The entity-relationship model—toward a unified view of data. *ACM Trans. Database Syst.*, 1(1):9–36, Mar. 1976.

[18] CloudLab. CloudLab. https://www.cloudlab.us/, 2026. Accessed: October 15, 2026.

[19] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szymaniak, C. Taylor, R. Wang, and D. Woodford. Spanner: Google's Globally Distributed Database. *ACM Trans. Comput. Syst.*, 31(3), Aug. 2013.

[20] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov, M. Hentschel, J. Huang, A. W. Lee, A. Motivala, A. Q. Munir, S. Pelley, P. Povinec, G. Rahn, S. Triantafyllis, and P. Unterbrunner. The Snowflake Elastic Data Warehouse. In *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, page 215–226, New York, NY, USA, 2016. Association for Computing Machinery.

[21] Databricks. Access control in Unity Catalog. https://docs.databricks.com/aws/en/data-governance/unity-catalog/access-control, 2026. Accessed: October 15, 2026.

[22] Databricks. Transactions. https://docs.databricks.com/aws/en/transactions/, 2026. Accessed: October 15, 2026.

[23] Delta Lake Project. Table utility commands. https://docs.delta.io/delta-utility/, 2026. Accessed: October 15, 2026.

[24] L. P. Deutsch and D. G. Bobrow. An efficient, incremental, automatic garbage collector. *Commun. ACM*, 19(9):522–526, Sept. 1976.

[25] Dijkstra, Edsger W. and Lamport, Leslie and Martin, A. J. and Scholten, C. S. and Steffens, E. F. M. On-the-fly garbage collection: an exercise in cooperation. *Commun. ACM*, 21(11):966–975, Nov. 1978.

[26] DuckDB Foundation. DuckLake. https://ducklake.select/, 2026. Accessed: October 15, 2026.

[27] P. Edara and M. Pasumansky. Big metadata: when metadata is big data. *Proc. VLDB Endow.*, 14(12):3083–3095, July 2021.

[28] P. Fuchs, D. Margan, and J. Giceva. Sortledton: a universal, transactional graph data structure. *Proc. VLDB Endow.*, 15(6):1173–1186, Feb. 2022.

[29] Google. Introduction to external tables. https://docs.cloud.google.com/bigquery/docs/external-tables, 2026. Accessed: October 15, 2026.

[30] Google Cloud. Bigtable. https://cloud.google.com/bigtable, 2026. Accessed: October 15, 2026.

[31] T. Götz, D. Ritter, M. El-Hindi, and J. Giceva. Interoperable ACID Transactions for Open Table Formats. *Proc. VLDB Endow.*, 19(11):3216–3230, Sept. 2026.

[32] gRPC Authors. gRPC. https://grpc.io/, 2026. Accessed: October 15, 2026.

[33] P. Jain, P. Kraft, C. Power, T. Das, I. Stoica, and M. Zaharia. Analyzing and Comparing Lakehouse Storage Systems. In *The Conference on Innovative Data Systems Research, CIDR*, 2023.

[34] JanusGraph Authors. Janus Graph. https://janusgraph.org/, 2026. Accessed: October 15, 2026.

[35] R. Jones, A. Hosking, and E. Moss. *The garbage collection handbook: the art of automatic memory management.* Chapman and Hall/CRC, 2023.

[36] J. Kim, J. Yu, J. Ahn, S. Kang, and H. Jung. Diva: Making MVCC systems htap-friendly. In *Proceedings of the 2022 International Conference on Management of Data*, pages 49–64, 2022.

[37] P.-Å. Larson, M. Zwilling, and K. Farlee. The Hekaton Memory-Optimized OLTP Engine. *IEEE Data Eng. Bull.*, 36(2):34–40, 2013.

[38] J. Levandoski, G. Casto, M. Deng, R. Desai, P. Edara, T. Hottelier, A. Hormati, A. Johnson, J. Johnson, D. Kurzyniec, S. McVeety, P. Ramanathan, G. Saxena, V. Shanmugan, and Y. Volobuev. BigLake: BigQuery's Evolution toward a Multi-Cloud Lakehouse. In *Companion of the 2024 International Conference on Management of Data*, SIGMOD '24, page 334–346, New York, NY, USA, 2024. Association for Computing Machinery.

[39] A. Lumsdaine, D. Gregor, B. Hendrickson, and J. Berry. Challenges in parallel graph processing. *Parallel Processing Letters*, 17(01):5–20, 2007.

[40] MariaDB Foundation. MariaDB. https://mariadb.org/, 2026. Accessed: October 15, 2026.

[41] Meta Platforms. RocksDB. https://rocksdb.org/, 2026. Accessed: October 15, 2026.

[42] Microsoft. Microsoft Fabric. https://www.microsoft.com/en-us/microsoft-fabric, 2026. Accessed: October 15, 2026.

[43] Microsoft. Microsoft SQL Server. https://www.microsoft.com/en-us/sql-server, 2026. Accessed: October 15, 2026.

[44] MongoDB. MongoDB. https://www.mongodb.com/, 2026. Accessed: October 15, 2026.

[45] A. Motivala. How FoundationDB Powers Snowflake Metadata Forward. https://www.snowflake.com/blog/how-foundationdb-powers-snowflake-metadata-forward/, 2018. Accessed: October 15, 2026.

[46] Neo4j. Neo4j. https://neo4j.com/, 2026. Accessed: October 15, 2026.

[47] Neon. Neon. https://neon.com/, 2026. Accessed: October 15, 2026.

[48] T. Neumann, T. Mühlbauer, and A. Kemper. Fast Serializable Multi-Version Concurrency Control for Main-Memory Database Systems. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, SIGMOD '15, page 677–689, New York, NY, USA, 2015. Association for Computing Machinery.

[49] K. Oh, P. Nilangekar, and A. Deshpande. TreeCat: Standalone Catalog Engine for Large Data Systems. *Proc. VLDB Endow.*, 18(11):4323–4336, July 2025.

[50] A. Okolnychyi, C. Sun, K. Tanimura, R. Spitzer, R. Blue, S. Ho, Y. Gu, V. Lakkundi, and D. Tsai. Petabyte-Scale Row-Level Operations in Data Lakehouses. *Proc. VLDB Endow.*, 17(12):4159–4172, Aug. 2024.

[51] Onehouse. Onehouse. https://www.onehouse.ai/, 2026. Accessed: October 15, 2026.

[52] Oracle. MySQL. https://www.mysql.com/, 2026. Accessed: October 15, 2026.

[53] Project Nessie. Project Nessie: Transactional Catalog for Data Lakes with Git-like semantics. https://projectnessie.org/, 2026. Accessed: October 15, 2026.

[54] S. Sakdeo. Open Sourcing OpenHouse: A Control Plane for Managing Tables in a Data Lakehouse. https://www.linkedin.com/blog/engineering/open-source/open-sourcing-openhouse, 2024. Accessed: October 15, 2026.

[55] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene, and A. Milovidov. ClickHouse - Lightning Fast Analytics for Everyone. *Proc. VLDB Endow.*, 17(12):3731–3744, Aug. 2024.

[56] R. Sethi, M. Traverso, D. Sundstrom, D. Phillips, W. Xie, Y. Sun, N. Yegitbasi, H. Jin, E. Hwang, N. Shingte, and C. Berner. Presto: SQL on Everything. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1802–1813, 2019.

[57] SingleStore. SingleStore. https://www.singlestore.com/, 2026. Accessed: October 15, 2026.

[58] Snowflake. Introducing Polaris Catalog: An Open Source Catalog for Apache Iceberg. https://www.snowflake.com/en/blog/introducing-polaris-catalog/, 2024. Accessed: October 15, 2026.

[59] Snowflake. Apache Iceberg tables. https://docs.snowflake.com/en/user-guide/tables-iceberg, 2026. Accessed: October 15, 2026.

[60] G. L. Steele. Multiprocessing compactifying garbage collection. *Commun. ACM*, 18(9):495–508, Sept. 1975.

[61] M. Stonebraker, D. Abadi, D. J. DeWitt, S. Madden, E. Paulson, A. Pavlo, and A. Rasin. MapReduce and parallel DBMSs: friends or foes? *Commun. ACM*, 53(1):64–71, Jan. 2010.

[62] S. Sumit, P. Wason, and S. Narayanan. Metadata-as-Data in Apache Hudi: A Multi-Modal Index Substrate for Lakehouse Tables. In *Proceedings of the 1st International Workshop on Data FORMATS for Modern Architectures and Workloads*, FORMATS '26, New York, NY, USA, 2026. Association for Computing Machinery.

[63] The Apache Software Foundation. Apache Hudi. https://hudi.apache.org/, 2026. Accessed: October 15, 2026.

[64] The Apache Software Foundation. Apache Hudi: Concurrency Control. https://hudi.apache.org/docs/concurrency_control/, 2026. Accessed: October 15, 2026.

[65] The Apache Software Foundation. Apache Iceberg. https://iceberg.apache.org/, 2026. Accessed: October 15, 2026.

[66] The Apache Software Foundation. Apache Impala. https://impala.apache.org/, 2026. Accessed: October 15, 2026.

[67] The Apache Software Foundation. Apache ORC. https://orc.apache.org/, 2026. Accessed: October 15, 2026.

[68] The Apache Software Foundation. Apache Parquet. https://parquet.apache.org/, 2026. Accessed: October 15, 2026.

[69] The Apache Software Foundation. Cassandra. https://cassandra.apache.org/, 2026. Accessed: October 15, 2026.

[70] The Apache Software Foundation. HBase. https://hbase.apache.org/, 2026. Accessed: October 15, 2026.

[71] The Linux Foundation. Delta Lake. https://delta.io/, 2026. Accessed: October 15, 2026.

[72] The PostgreSQL Global Development Group. 24.1 Routine Vacuuming. https://www.postgresql.org/docs/current/routine-vacuuming.html, 2026. Accessed: October 15, 2026.

[73] The PostgreSQL Global Development Group. PostgreSQL. https://www.postgresql.org/, 2026. Accessed: October 15, 2026.

[74] A. Thusoo, J. S. Sarma, N. Jain, Z. Shao, P. Chakka, N. Zhang, S. Antony, H. Liu, and R. Murthy. Hive - a petabyte scale data warehouse using Hadoop. In *2010 IEEE 26th International Conference on Data Engineering (ICDE 2010)*, pages

996–1005, 2010.

[75] TigerGraph. TigerGraph. https://www.tigergraph.com/, 2026. Accessed: October 15, 2026.

[76] A. Verbitski, A. Gupta, D. Saha, M. Brahmadesam, K. Gupta, R. Mittal, S. Krishnamurthy, S. Maurice, T. Kharatishvili, and X. Bao. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *Proceedings of the 2017 ACM International Conference on Management of Data*, SIGMOD '17, page 1041–1052, New York, NY, USA, 2017. Association for Computing Machinery.

[77] R. Xin. The Three Golden Ages of Database Engineering: From SIGMOD '85 to the Agentic Era. Keynote address at the 52nd International Conference on Very Large Data Bases (VLDB), September 2026.

[78] T. Yuasa. Real-time garbage collection on general-purpose machines. *Journal of Systems and Software*, 11(3):181–198, 1990.

[79] J. Zhou, M. Xu, A. Shraer, B. Namasivayam, A. Miller, E. Tschannen, S. Atherton, A. J. Beamon, R. Sears, J. Leach, D. Rosenthal, X. Dong, W. Wilson, B. Collins, D. Scherer, A. Grieser, Y. Liu, A. Moore, B. Muppana, X. Su, and V. Yadav. FoundationDB: A Distributed Unbundled Transactional Key Value Store. In *Proceedings of the 2021 International Conference on Management of Data*, SIGMOD '21, page 2653–2666, New York, NY, USA, 2021. Association for Computing Machinery.

[80] X. Zhu, G. Feng, M. Serafini, X. Ma, J. Yu, L. Xie, A. Aboulnaga, and W. Chen. LiveGraph: a transactional graph storage system with purely sequential adjacency list scans. *Proc. VLDB Endow.*, 13(7):1020–1034, Mar. 2020.